```
xdebug.client_host = 192.168.10.1
xdebug.mode = debug
xdebug.start_with_request = yes
```

Use the xphp shell alias on your Homestead virtual machine to debug a PHP CLI application:

```
xphp /path/to/script
```

Conclusion

In this chapter, we covered the essential aspects of testing and debugging your Laravel application. The chapter begins by introducing the testing features of Laravel, including the preconfigured phpunit.xml file and the helper functions that facilitate testing. It discusses the testing structure, including the Feature and Unit test folders, and emphasizes the importance of testing different aspects of the application. The chapter delves into the setup environment for testing, the **`.env.testing`** file, and the **`CreatesApplication`** trait.

The chapter explains the testing lifecycle in Laravel, covering the creation and execution of tests, reporting, profiling, and parallel execution. It discusses HTTP testing, including making requests, customizing headers, dealing with cookies, and authentication. Furthermore, it provides insights into debugging responses, handling exceptions, and testing file uploads. The chapter concludes by explaining how to use Xdebug for step debugging in the Laravel Homestead environment.

Throughout the chapter, various code examples and practical tips are provided to help readers understand and implement testing and debugging techniques effectively in their Laravel applications. The chapter aims to equip developers with the necessary knowledge and tools to create robust and well-tested Laravel applications.

In the next chapter, we will be looking at the mail and notifications feature of Laravel that provides this feature to the APIs.

Points to Remember

- Testing a Laravel application involves a structured approach, distinguishing between Unit and Feature tests. Unit tests focus on